

University of Wrocław

UWr team

Adam Biel, Mateusz Jurach, Karol Żeleźnik

2026-10-18

1 Templatki 1

2 Wzorki 1

3 Matma 1

4 Struktury danych 4

5 Grafy 7

6 Tekstówki 8

7 Flow 8

8 Bignumy 10

Templatki (1)

.bashrc

11 lines

```
c() {
  g++ -std=c++20 -Wall -Wno-sign-conversion -Wextra -Wshadow -
      Wconversion -Wfloat-equal \
  -fsanitize=address,undefined -DLOCAL -DDEBUG -D_GLIBCXX_DEBUG
       -ggdb3 $1.cpp -o $1
}

nc() {
  g++ -std=c++20 -DLOCAL -O3 -static $1.cpp -o $1
}

alias cp='cp -i'
alias mv='mv -i'
```

.vimrc

7 lines

```
set nocompatible
filetype indent on
set autoindent
set number
syntax on
set ts=4 sw=4
set rnu
```

main.cpp

24 lines

```
#include "bits/stdc++.h"
using namespace std;
#ifdef DEBUG
auto&operator<<(auto &o, pair<auto, auto> p) {o << "(" << p.
    first << ", " << p.second << ")"; return o;}
auto operator<<(auto&o,auto x)->decltype(x.end(),o){o<<"{"; for
    (auto e : x) o<<e<<", "; return o<<"}";}
#define debug(X) cerr << "["#X"]: " << X << '\n';
#else
#define cerr if(0)cout
#define debug(X) ;
#endif
using ll = long long;
#define all(v) (v).begin(), (v).end()
#define ssize(x) int(x.size())
#define fi first
#define se second
#define mp make_pair
#define eb emplace_back

int main() {
  ios_base::sync_with_stdio(false); cin.tie(nullptr);

  return 0;
}
```

gen.cpp

Opis: Dodatek do generowania testów, lub innego losowania. Ziarno można zmieniać np. `rng.seed(atoi(argv[1]));`

```
mt19937 rng(random_device{}());
int rd(int l, int r) {
  return uniform_int_distribution<int>(l, r)(rng);
}
```

Wzorki (2)

2.1 Fibonacci

$f_0 = 0$, $f_1 = 1$, $f_n = f_{n-1} + f_{n-2}$, dla $n \geq 2$.

$$f_n = \frac{\alpha^n - \beta^n}{\sqrt{5}} = \frac{\alpha^n - \beta^n}{\alpha - \beta},$$

dla $\alpha = \frac{1+\sqrt{5}}{2}$, $\beta = \frac{1-\sqrt{5}}{2}$. Jako, że $|\beta| < 1$ to

$$f_n \approx \frac{1}{\sqrt{5}}\alpha^n.$$

Niech $F(x) = \sum_{n=0}^{\infty} f_n x^n$, wtedy

$$F(x) = \frac{x}{1 - x - x^2}.$$

$$f_1 + f_2 + \cdots + f_n = f_{n+2} - 1$$

$$\mathrm{NWD}(f_n, f_m) = f_{\mathrm{NWD}(n,m)}.$$

2.2 Liczby Catalana

$$C_n = \frac{1}{n+1}\binom{2n}{n}$$

$$C_n = C_0 C_{n-1} + C_1 C_{n-2} + \ldots + C_{n-2} C_1 + C_{n-1} C_0$$

$$C_{n+1} = \frac{2\,(2n+1)}{n+2} C_n$$

2.3 Funkcje tworzące

$$\frac{1}{1-ax} = \sum a^n x^n$$

$$F(x) = \frac{x}{1 - x - x^2}$$

$$C(x) = \frac{1 - \sqrt{1-4x}}{2x}$$

$$\exp(x) = \sum_{n \geq 0} \frac{x^n}{n!}$$

$$-\ln(1-x) = \sum_{n \geq 1} \frac{x^n}{n}$$

$$\frac{1}{(1-x)^k} = \sum_{n \geq 0} \binom{n+k-1}{n} x^n$$

$$\prod_{n \geq 1} (1 - x^n) = 1 + \sum_{k \geq 1} (-1)^k \left(x^{\frac{k(3k+1)}{2}} + x^{\frac{k(3k-1)}{2}} \right)$$

2.4 Geometria

2.4.1 Obrót płaszczyzny wokół punktu $(0, 0)$.

$$R_\theta \left(\begin{bmatrix} x \\ y \end{bmatrix} \right) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}.$$

W szczególności

$$R_{\frac{\pi}{4}} \left(\begin{bmatrix} x \\ y \end{bmatrix} \right) = \frac{1}{\sqrt{2}} \begin{bmatrix} x - y \\ x + y \end{bmatrix}.$$

2.5 Zasada włączeń i wyłączeń

$$|A_1 \cup A_2 \cup \cdots \cup A_n| = \sum_{i_1, i_2, \ldots, i_k} (-1)^k \left| A_{i_1} \cap A_{i_2} \ldots A_{i_k} \right|$$

2.6 Funkcje arytmetyczne

$$\varphi(ab) = \varphi(a) \cdot \varphi(b) \cdot \frac{d}{\varphi(d)},$$

gdzie $d = \mathrm{NWD}(a, b)$. Jeżeli $n = p_1^{\alpha_1} p_2^{\alpha_2} \ldots p_k^{\alpha_k}$, to

$$\varphi(n) = n \cdot \left(1 - \frac{1}{p_1}\right) \cdot \left(1 - \frac{1}{p_2}\right) \ldots \left(1 - \frac{1}{p_k}\right).$$

$$\varphi(n) = \sum_{d | n} \varphi(d)$$

Niech $\phi(n) = \sum_{i=1}^{n} \varphi(i)$, wtedy

$$\phi(n) = \frac{n(n+1)}{2} - \sum_{i=2}^{n} \phi\left(\left\lfloor \frac{n}{i} \right\rfloor\right)$$

Matma (3)

ext-euclid.cpp

Opis: $O(\log n)$. Zwraca trójkę [d, x, y], że $d = \text{NWD}(a, b)$ i $ax + by = d$.

```
tuple<ll, ll, ll> euclid(ll a, ll b) {
  if (b == 0) return make_tuple(a, 1, 0);
  auto [d, x, y] = euclid(b, a%b);
  return make_tuple(d, y, x - (a/b) * y);
}
```

phi.cpp

Opis: Liczy $\phi(n)$ w $O(\sqrt{n})$.

```
int phi(int n) {
    int result = n;
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            while (n % i == 0)
                n /= i;
            result -= result / i;
        }
    }
    if (n > 1)
        result -= result / n;
    return result;
}
```

phi-to-n.cpp

Opis: Liczy $\varphi(a)$ dla każdego $a \leq n$ w $O(n \log \log n)$.

```
void phi_1_to_n(int n) {
    vector<int> phi(n + 1);
    for (int i = 0; i <= n; i++)
        phi[i] = i;

    for (int i = 2; i <= n; i++) {
        if (phi[i] == i) {
            for (int j = i; j <= n; j += i)
                phi[j] -= phi[j] / i;
        }
    }
}
```

sum-phi.cpp

Opis: $O(n^{2/3})$ dla $HI = n^{2/3}$. Liczy $\sum_{i=1}^{n} \varphi(n)$.

```
struct sieve {
  vector<int> phi;
  sieve(int n) {
    phi.resize(n+1);
    for (int i = 0; i <= n; ++i) phi[i] = i;

    for (int i = 2; i <= n; ++i) {
      if (phi[i] == i) {
        for (int j = i; j <= n; j += i)
          phi[j] -= phi[j]/i;
      }
    }
  }
};

const int HI = 1'000'003;
sieve S(HI);

unordered_map<int, int> mem;
vector<int> small(HI);

int sum_phi(int n) {
  if (n < HI) return small[n];
  if (mem[n] > 0) return mem[n];

  int res = ll(n) * (n+1) / 2 % mod;
  for (int l = 2, r; l <= n; l = r) {
    int k = n / l;
    r = n / k + 1;
    res = sub(res, mul((r - l), sum_phi(k)));
  }
  return mem[n] = res;
}
```

crt.cpp

Opis: $O(\log n)$. Zwraca takie x, że $x \equiv a \mod m$ i $x \equiv b \mod n$. Mogą nie być względnie pierwsze ale wtedy assert wywali.

```
ll crt(ll a, ll m, ll b, ll n) {
  if(n > m) swap(a, b), swap(m, n);
  auto [d, x, y] = euclid(m, n);
  assert((a - b) % d == 0);
  ll ret = (b - a) % n * x % n / d * m + a;
  return ret < 0 ? ret + m * n / d : ret;
}
```

xor-base.cpp

Opis: $O(nB)$ gdzie B to liczba bitów. Zwraca minimalny zbiór b taki że każdy element z x można zapisać jako xor pewnych elementów b.

```
vector<ll> xor_base(const vector<ll> &x) {
  vector<ll> basis;
  for (ll a: x) {
    ll A = a;
    for (ll b: basis) A = min(A,A^b);
    if (A) basis.push_back(A);
  }
  return basis;
}
```

fft.cpp

Opis: Mnożenie wielomianów w $O(n \log n)$.

```
using ld = long double;
using cd = complex<long double>;
const double PI = acos(-1);

void fft(vector<cd> &a, int n, bool invert = false) {
  assert((n&(n-1)) == 0);
  a.resize(n);
  for (int i = 1, j = 0; i < n; ++i) {
    int bit = n/2;
    for ( ; bit&j; bit /= 2)
      j ^= bit;
    j ^= bit;

    if (i < j) swap(a[i], a[j]);
  }

  for (int len = 2; len <= n; len *= 2) {
    ld ang = 2 * PI / len * (invert ? -1 : 1);
    cd r_len(cos(ang), sin(ang));
    for (int i = 0; i < n; i += len) {
      cd r(1);
      for (int j = 0; j < len/2; ++j, r *= r_len) {
        cd u = a[i + j], v = r * a[i + j + len/2];
        a[i+j] = u + v;
        a[i+j+len/2] = u - v;
      }
    }
  }
  if (invert)
    for (cd &x : a) x /= n;
}

vector<int> conv(vector<int> a, vector<int> b) {
  if (a.empty() || b.empty()) return {};

  vector<cd> fa(all(a)), fb(all(b));
  int n = ssize(a) + ssize(b) - 1;
  int sz = 1;
  while (sz < ssize(a) + ssize(b) - 1) sz *= 2;
  fft(fa, sz);
  fft(fb, sz);
  for (int i = 0; i < sz; ++i) fa[i] *= fb[i];
  fft(fa, sz, true);

  vector<int> result(n);
  for (int i = 0; i < n; ++i)
    result[i] = round(fa[i].real());
  return result;
}
```

mod.cpp

Opis: operacje mod 998244353.

```
const int mod = 998244353;

int add(int a, int b) {
  a += b;
  return a >= mod ? a-mod : a;
}

int sub(int a, int b) {
  return add(a, mod-b);
}

int mul(int a, int b) {
  return int(((ll)a*b)%mod);
}

int fpow(int a, int b) {
  int res = 1;
  while (b) {
    if (b&1) {
      res = mul(a, res);
      b--;
    }
    a = mul(a, a);
    b /= 2;
  }
  return res;
}

int inv(int x) {
  return fpow(x, mod-2);
}
```

ntt.cpp

Opis: $O(n \log n)$. Mnożenie wielomianów mod 998244353.

```
#include "mod.cpp" //keep-include

const int g = 3;
const int g_1 = 332748118;

using vi = vector<int>;

void ntt(vi &a, int n, bool invert = false) {
```

```
  assert((n&(n-1)) == 0);
  a.resize(n);
  for (int i = 1, j = 0; i < n; ++i) {
    int bit = n/2;
    for ( ; bit&j; bit /= 2)
      j ^= bit;
    j ^= bit;

    if (i < j) swap(a[i], a[j]);
  }

  for (int len = 2; len <= n; len *= 2) {
    int r_len = fpow(invert ? g_1 : g, (mod-1) / len);
    for (int i = 0; i < n; i += len) {
      int r = 1;
      for (int j = 0; j < len/2; ++j, r = mul(r, r_len)) {
        int u = a[i + j], v = mul(r, a[i + j + len/2]);
        a[i+j] = add(u, v);
        a[i+j+len/2] = sub(u, v);
      }
    }
  }

  if (invert) {
    int inv_n = inv(n);
    for (int &x : a) x = mul(x, inv_n);
  }
}

vi conv(vi a, vi b) {
  if (a.empty() || b.empty()) return {};
  int n = ssize(a) + ssize(b) - 1;
  int sz = 1;
  while (sz < ssize(a) + ssize(b) - 1) sz *= 2;
  ntt(a, sz);
  ntt(b, sz);
  for (int i = 0; i < sz; ++i) a[i] = mul(a[i], b[i]);
  ntt(a, sz, true);
  a.resize(n);
  return a;
}
```

poly.cpp

Opis: Operacje na wielmianach mod 998244353. Można przepisać co się chce, przy funkcjach jest napisane co potrzeba. deriv, integr w $O(n)$. inv, log, exp w $O(n \log n)$.

```
#include "ntt.cpp" //keep-include

using vi = vector<int>;
vi mod_xn(const vi &v, int n) { //ZAWSZE
  return vi(v.begin(), v.begin() + min(n, ssize(v)));
}

vi add(vi a, const vi &b) {
  a.resize(max(ssize(a), ssize(b)));
  for (int i = 0; i < ssize(b); ++i)
    a[i] = add(a[i], b[i]);
  return a;
}

vi sub(vi a, const vi &b) { //ZAWSZE
  a.resize(max(ssize(a), ssize(b)));
  for (int i = 0; i < ssize(b); ++i)
    a[i] = sub(a[i], b[i]);
  return a;
}

vi deriv(vi p) {
  for (int i = 1; i < ssize(p); ++i) p[i] = mul(p[i], i);
  if (ssize(p)) p.erase(p.begin());
  return p;
}

vi integr(vi p) {
  for (int i = 0; i < ssize(p); ++i)
    p[i] = mul(p[i], inv(i+1));
  p.insert(p.begin(), 0);
  return p;
}

vi inv(vi p, int n) {
  assert(ssize(p) && p[0] != 0);
  if (n == 1)
    return {inv(p[0])};
  p.resize(n);
  vi _p = p;
  for (int i = 1; i < n; i += 2) _p[i] = sub(0, p[i]);

  vi t = mod_xn(conv(p, _p), n);
  for (int i = 1; i < n; i += 2) assert(t[i] == 0);
  for (int i = 0; i < n; i += 2)
    t[i/2] = t[i];
  t = inv(t, (n+1)/2);
  t.resize(n);
  for (int i = (n-1)/2; i >= 0; --i) {
    t[2*i] = t[i];
    if(i) t[i] = 0;
  }
  return mod_xn(conv(_p, t), n);
}

vi log(const vi &p, int n) { //DERIV, INV, INTEGR
  assert(ssize(p) && p[0] == 1);
  return integr(mod_xn(conv(deriv(mod_xn(p, n)), inv(p, n)), n
      -1));
}

vi exp(const vi &p, int n) { //ADD, LOG
  assert(p.empty() || p[0] == 0);
  vi q = {1};
  for (int x = 1; x < n; x *= 2)
    q = mod_xn(conv(q, add(sub({1}, log(q, 2*x)), mod_xn(p, 2*x
        ))), 2*x);
  q.resize(n);
  return q;
}

vi powi(vi p, int k, int n) { //LOG, EXP
  p = mod_xn(p, n);
  int cnt = 0;
  while(cnt < ssize(p) and !p[cnt])
    ++cnt;
  if(ll(cnt) * k >= n)
    return {};
  p.erase(p.begin(), p.begin() + cnt);
  if(p.empty())
    return k ? vi{} : vi{1};
  int powi0 = fpow(p[0], k);
  int inv0 = inv(p[0]);
  for (int &x : p) x = mul(x, inv0);
  p = log(p, n - cnt * k);
  for (int &x : p) x = mul(x, k);
  p = exp(p, n - cnt * k);
  for (int &x : p) x = mul(x, powi0);
  vi t(cnt * k, 0);
  p.insert(p.begin(), t.begin(), t.end());
  return p;
}

vi sqrt(const vi &p, int n) { //POW
  assert(ssize(p) && p[0] == 1);
  return powi(p, inv(2), n);
}
```

poly2d.cpp

Opis: Operacje na wielmianach dwóch zmiennych mod 998244353. Mnożenie w $O(mn \log(m+n))$. Dzielenie zwraca vector współczynników przy x^{n-1}, w czasie $O(mn \log^2(n+m))$.

```
#include "poly.cpp" //keep-include

void ntt(vector<vi> &a, int n, int m, bool inverse = false) {
  a.resize(n);
  for (int i = 0; i < ssize(a); ++i)
    ntt(a[i], m, inverse);
  for (int j = 0; j < ssize(a[0]); ++j) {
    vi c;
    for (int i = 0; i < ssize(a); ++i)
      c.eb(a[i][j]);
    ntt(c, n, inverse);
    for (int i = 0; i < ssize(a); ++i)
      a[i][j] = c[i];
  }
}

vector<vi> conv(vector<vi> a, vector<vi> b, int n, int m) {
  if (a.empty() || b.empty()) return {};
  int l1 = ssize(a) + ssize(b) - 1, sz1 = 1 << __lg(2*l1-1);
  int l2 = ssize(a[0]) + ssize(b[0]) - 1, sz2 = 1 << __lg(2*l2
      -1);

  ntt(a, sz1, sz2);
  ntt(b, sz1, sz2);
  for (int i = 0; i < sz1; ++i)
    for (int j = 0; j < sz2; ++j) a[i][j] = mul(a[i][j], b[i][j
        ]);
  ntt(a, sz1, sz2, true);
  a.resize(n);
  for (int i = 0; i < n; ++i) a[i].resize(m);
  return a;
}

vi div(vector<vi> b, vector<vi> a, int n, int m) {
  if (n == 1) {
    return conv(b[0], inv(a[0], m));
  }
  vector<vi> q = a;
  for (int i = 1; i < n; i += 2)
    for (int j = 0; j < ssize(a[i]); ++j) q[i][j] = sub(0, q[i
        ][j]);

  vector<vi> v = conv(a, q, n, 2*m);
  debug(v);
  q = conv(b, q, n, 2*m);
  for (int i = 0; i < ssize(v); i += 2) {
    for (int j = 0; j < ssize(v[i]); ++j) {
      v[i/2][j] = v[i][j];
    }
  }
  v.resize((n+1)/2);

  if (n%2 == 0) {
    for (int i = 1; i < n; i += 2)
      for (int j = 0; j < 2*m; ++j)
        q[i/2][j] = q[i][j];
    q.resize(n/2);
```

```
    return div(q, v, n/2, 2*m);
  }
  else {
    for (int i = 0; i < n; i += 2)
      for (int j = 0; j < 2*m; ++j)
        q[i/2][j] = q[i][j];
    q.resize((n+1)/2);
    return div(q, v, (n+1)/2, 2*m);
  }
}
```

lis.cpp

Opis: $O(n \log n)$. Zwraca długość najdłuższego podciągu rosnącego.

```
int LIS(vector<int> A)
{
  vector<int> vec;
  for(auto& v : A)
    if(vec.size() == 0 || v > vec.back())
      vec.push_back(v);
    else
      *lower_bound(vec.begin(), vec.end(), v) = v;
  return vec.size();
}
```

mo.cpp

Opis: $O(1)$. Sort do MO. Sortuje przedziały $[a.x, a.y]$.

```
const int k = 500;
sort(events.begin(), events.end(), [&](type a, type b){
    if(a.x/k < b.x/k) return true;
    if(a.x/k > b.x/k) return false;
    if((a.x/k)%2 == 1)
    return a.y < b.y;
    else
    return a.y > b.y;
  });
```

sos.cpp

Opis: $O(l \cdot 2^l)$. Zwraca $\mathbf{sos}[mask]$ równe sumie $\mathbf{s}[m]$ po podzbiorach $mask$.

```
vector<int> SOS(int l, vector<int>& s)
{
  vector<int> sos(1<<l);
  for(int mask = 0;mask<(1<<l);mask++)
    sos[mask] = s[mask];
  for(int i=0;i<l;i++)
    for(int mask=0;mask<(1<<l);mask++)
      if((mask & (1<<i)) != 0)
          sos[mask] += sos[mask^(1<<i)];

  return sos;
}
```

or-convolution.cpp

Opis: $O(n \log n)$. Oblicza tablicę $c_k = \sum_{i|j=k} a_i \cdot b_j$.

```
void sos(vector<ll> &s, bool inverse = false) {
  int n = ssize(s);
  int l = 0;
  int pot = 1;
  while (2*pot <= n) {
    pot *= 2;
    l++;
  }
  for(int i=0;i<l;i++)
    for(int mask=0;mask<n;mask++)
      if((mask & (1<<i)) != 0)
        s[mask] += (inverse ? -s[mask^(1<<i)] : s[mask^(1<<i)])
            ;
}

vector<int> or_convolution(vector<int> a, vector<int> b) {
  int sz = max(ssize(a), ssize(b));
  a.resize(sz); b.resize(sz);
  vector<int> c(sz);

  sos(a); sos(b);
  for (int i = 0; i < sz; ++i) c[i] = a[i] * b[i];
  sos(c, true);

  return c;
}
```

and-convolution.cpp

Opis: $O(n \log n)$. Oblicza tablicę $c_k = \sum_{i\&j=k} a_i \cdot b_j$.

```
#include "or_convolution.cpp" //keep-include

vector<int> and_convolution(vector<int> a, vector<int> b) {
  int n = max(ssize(a), ssize(b));
  int sz = 1;
  while (sz < n) sz *= 2;
  a.resize(sz); b.resize(sz);

  for (int i = 0; i < sz/2; ++i) {
    swap(a[i], a[(sz-1)^i]);
    swap(b[i], b[(sz-1)^i]);
  }

  vector<int> c = or_convolution(a, b);

  for (int i = 0; i < sz/2; ++i) {
    swap(c[i], c[(sz-1)^i]);
  }

  return c;
}
```

xor-convolution.cpp

Opis: $O(n \log n)$. Oblicza tablicę $c_k = \sum_{i\wedge j=k} a_i \cdot b_j$.

```
#include "mod.cpp" //keep-include

void transform(vector<int> &a, bool inverse = false) {
  int m = ssize(a);
  for (int n = (inverse ? 1 : m / 2); inverse ? n < m : n > 0;
       inverse ? n *= 2 : n /= 2)
    for (int i = 0; i < m; i += 2 * n)
      for (int j = 0; j < n; j++) {
        int x = a[i + j], y = a[i + j + n];
        a[i + j] = add(x, y);
        a[i + j + n] = sub(x, y);
      }
}

vector<int> xor_convolution(vector<int> a, vector<int> b) {
  int sz = max(ssize(a), ssize(b));
  a.resize(sz); b.resize(sz);
  vector<int> c(sz);

  transform(a); transform(b);
  for (int i = 0; i < sz; ++i) c[i] = mul(a[i], b[i]);
  transform(c, true);

  int mrev = inverse(m);
  for (int &x : c)
    x = mul(x, mrev);

  return c;
}
```

gcd-convolution.cpp

Opis: $O(n \log n)$. Oblicza tablicę $c_k = \sum_{\mathrm{NWD}(i,j)=k} a_i \cdot b_j$.

```
#include "mod.cpp" //keep-include

vector<int> gcd_convolution(vector<int> a, vector<int> b) {
  int sz = max(ssize(a), ssize(b));
  a.resize(sz); b.resize(sz);
  for (int i = 1; i < sz; ++i) {
    for (int j = 2*i; j < sz; j += i) {
      a[i] = add(a[i], a[j]);
      b[i] = add(b[i], b[j]);
    }
  }

  vector<int> c(sz);
  for (int i = 0; i < sz; ++i) c[i] = mul(a[i], b[i]);

  for (int i = sz-1; i >= 1; --i)
    for (int j = 2*i; j < sz; j += i)
      c[i] = sub(c[i], c[j]);

  return c;
}
```

Struktury danych (4)

min-que.cpp

Opis: Zamortyzowane $O(n)$. Dodawanie do końca, usuwanie z przodu i podaj min.

```
struct minQue
{
  deque<pair<int, int> > q;
  int _size = 0;
  void push(int val)
  {
    _size++;
    pair<int, int> nq = {val, 1};
    while(!q.empty() && q.back().first >= val) //zmienic na <=
        jezeli ma byc maxQue
    {
      nq.second += q.back().second;
      q.pop_back();
    }
    q.push_back(nq);
  }
  void pop()
  {
    _size--;
    assert(!q.empty());
    if(q.front().second > 1) q.front().second--;
    else q.pop_front();
  }
  int top()
  {
    return q.front().first;
  }
  int size()
  {
    return _size;
  }
};
```

treap.cpp

Opis: $O(\log n)$ czasowo. Treap z operacjami na (multi)secie. `kth(t, k)` zwraca k-tą najmniejszą liczbę.

```
namespace Treap {
  mt19937 rng(420);
  struct Node {
    Node *left, *right;
    int val, rank;
    int size;
    Node(): left(nullptr), right(nullptr), val(0), rank(int(rng
        ())), size(0) {}
  };
  using pNode = Node*;

  Node pool[1'000'003];
  int curr = 0;

  int sz(pNode t) {
    return t == nullptr ? 0 : t->size;
  }

  void upd(pNode t) {
    t->size = sz(t->left) + sz(t->right) + 1;
  }

  pNode merge(pNode a, pNode b) {
    if(a == nullptr) return b;
    if(b == nullptr) return a;

    if(a->rank < b->rank) {
      a->right = merge(a->right, b);
      upd(a);
      return a;
    }
    else {
      b->left = merge(a, b->left);
      upd(b);
      return b;
    }
  }

  pair<pNode, pNode> split(pNode t, int x) {
    if(t == nullptr) return {nullptr, nullptr};

    if(t->val <= x) {
      auto [a, b] = split(t->right, x);
      t->right = a;
      upd(t);
      return {t, b};
    }
    else {
      auto [a, b] = split(t->left, x);
      t->left = b;
      upd(t);
      return {a, t};
    }
  }

  void insert(pNode &root, int x) {
    pNode u = pool + (curr++);
    u->val = x;
    u->size = 1;
    auto [a, b] = split(root, x);
    root = merge(merge(a, u), b);
  }

  bool find(pNode t, int x) {
    if(t == nullptr) return false;
    if(x == t->val) return true;
    if(x < t->val) return find(t->left, x);
    else return find(t->right, x);
  }

  void erase(pNode root, int x) {
    auto [ab, c] = split(root, x);
    auto [a, b] = split(ab, x-1);
    root = merge(a, c);
  }

  int kth(pNode t, int k) {
    assert(k >= 0);
    if(sz(t->left) == k-1) return t->val;
    if(k > sz(t->left)) return kth(t->right, k-1-sz(t->left));
    else return kth(t->left, k);
  }
}
```

implicit-treap.cpp

Opis: $O(\log n)$ czasowo. Implicit treap z podstawowymi operacjami, przykładowo odwracanie na przedziale (moze być dowolna operacja). `insert(root, k, x)` wstawia element o wartości x w k-te miejsce tablicy. `rev(root, l, r)` odwraca przedział $[l; r]$ (indeksy od 0).

```
namespace Treap {
  mt19937 rng(420);
  struct Node {
    Node *left, *right;
    int rank;
    int size;
    bool rev;
    int val;
    Node(): left(nullptr), right(nullptr), rank(int(rng())),
        size(0), rev(false), val(0) {}
  };
  using pNode = Node*;

  Node pool[300'003];
  int curr = 0;

  int sz(pNode t) {
    return t == nullptr ? 0 : t->size;
  }

  void upd(pNode t) {
    t->size = sz(t->left) + sz(t->right) + 1;
  }

  void push_lazy(pNode t) {
    if(t->rev == false) return;
    if(t->left) t->left->rev = !t->left->rev;
    if(t->right) t->right->rev = !t->right->rev;
    swap(t->left, t->right);
    t->rev = false;
  }

  pNode merge(pNode a, pNode b) {
    if(a == nullptr) return b;
    if(b == nullptr) return a;

    push_lazy(a);
    push_lazy(b);
    if(a->rank < b->rank) {
      a->right = merge(a->right, b);
      upd(a);
      return a;
    }
    else {
      b->left = merge(a, b->left);
      upd(b);
      return b;
    }
  }

  pair<pNode, pNode> split(pNode t, int x) {
    if(t == nullptr) return {nullptr, nullptr};

    push_lazy(t);
    if(sz(t->left)+1 <= x) {
      auto [a, b] = split(t->right, x-(sz(t->left)+1));
      t->right = a;
      upd(t);
      return {t, b};
    }
    else {
      auto [a, b] = split(t->left, x);
      t->left = b;
      upd(t);
      return {a, t};
    }
  }

  pNode build(vector<int> a) {
    pNode root = nullptr;
    int n = ssize(a);
    for(int i = 0; i < n; ++i) {
      pNode u = pool + i;
      u->size = 1;
      u->val = a[i];
      root = merge(root, u);
    }
    curr += n;
    return root;
  }

  void insert(pNode &root, int k, int x) {
    auto [a, b] = split(root, k-1);
    Node *u = pool + (curr++);
    u->size = 1;
    u->val = x;
    root = merge(merge(a, u), b);
  }

  void reverse(pNode &root, int l, int r) {
    auto [ab, c] = split(root, r+1);
    auto [a, b] = split(ab, l);
    b->rev = !b->rev;
    root = merge(a, merge(b, c));
  }

  void print(pNode t) {
    if(t == nullptr) return;
    push_lazy(t);
    print(t->left);
    cout << t->val << ' ';
    print(t->right);
  }
}
```

uf-rollbacks.cpp

Opis: $O(\log n)$ czasowo. Union find z roll backami – `roll_back(x)` cofa ostatine x operacji unionn (union wierzchołków z tego samego zbioru się nie liczy).

```
struct dynamicUF {
  vector<int> e;
  dynamicUF(int n):e(n, -1) {}
```

```
  stack<array<int, 3>> s;

  int get(int a) {
    while(e[a] >= 0) a = e[a];
    return a;
  }

  int unionn(int a, int b) {
    a = get(a); b = get(b);
    if(a == b) return 0;
    if(-e[a] > -e[b]) swap(a, b);
    s.push({a, b, e[a]});
    e[b] += e[a];
    e[a] = b;
    return 1;
  }

  void roll_back(int m = 1) {
    while (m--) {
      auto [a, b, sz] = s.top();
      s.pop();
      e[b] -= sz;
      e[a] = sz;
    }
  }
};
```

fenwick-tree.cpp

Opis: (Binary Indexed Tree) $O(\log N)$, indeksowane od 0. Update w punkcie, zapytanie na przedziale

```
struct BIT {
  vector<int> tree;
  int N;
  BIT() {N = 0;}
  BIT(int n) {
    N = n;
    tree.resize(N);
  }
  void update(int p, int x) {
    for (++p; p <= N; p += (p&-p)) tree[p-1] += x;
  }
  int sum(int r) {
    int res = 0;
    for (; r > 0; r -= (r&-r)) res += tree[r-1];
    return res;
  }
  int query(int l, int r) {return sum(r+1)-sum(l);}
  int lower_bound(int sum) {
    int pos = 0;
    for (int pw = 1<<25; pw; pw>>=1) {
      int npos = pos+pw;
      if (npos <= N && tree[npos-1] < sum) {
        pos = npos; sum -= tree[pos-1];
      }
    }
    return pos;
  }
};
```

range.cpp

Opis: $O(\log n)$. Dodaj na przedziale, podaj wartość w punkcie.

```
#include "fenwick-tree.cpp" //keep-include

struct RangeAdd {
  BIT diff;
  RangeAdd(int n): diff(n) {}
  void update(int l, int r, int x) {
    if (l) diff.update(l-1, x);
    diff.update(r, -x);
  }

  int query(int p) {
    return diff.sum(p);
  }
};
```

fenwick-tree2d.cpp

Opis: $O(\log^2 n)$. Dodaj w punkcie, suma na prostokącie.

```
struct BIT2d {
  vector<vector<int>> tree;
  int n, m;
  BIT2d() {n = m = 0;}
  BIT2d(int N, int M) {
    n = N; m = M;
    tree.resize(n, vector<int>(m));
  }
  void update(int x, int y, int v) {
    for (++x; x <= n; x += (x&-x))
      for (int p = y+1; p <= m; p += (p&-p))
        tree[x-1][p-1] += v;
  }
  int sum(int x, int y) {
    int res = 0;
    for (; x; x -= (x&-x))
      for (int j = y; j; j -= (j&-j))
        res += tree[x-1][j-1];
    return res;
  }
  int query(int x1, int y1, int x2, int y2) {return sum(x2+1,
      y2+1)-sum(x1, y2+1)-sum(x2+1, y1)+sum(x1, y1);}
};
```

pbds.cpp

Opis: Ordered set.

```
#include <ext/pb_ds/assoc_container.hpp> //keep-include
#include <ext/pb_ds/tree_policy.hpp> //keep-include
using namespace __gnu_pbds;

template<typename T> using ordered_set = tree<
T,
null_type,
less<T>,
rb_tree_tag,
tree_order_statistics_node_update>;

void test() {
  ordered_set<int> S;
  S.insert(1);
  cout << *S.find_by_order(0) << '\n'; //zwraca k-ty element (
      indeksowane od 0)
  cout << (S.end() == find_by_order(1)) << '\n';
  cout << S.order_of_key(4) << '\n'; //zwraca liczbe elementow
      mniejszych od x
}
```

merge-sort-tree.cpp

Opis: Preprocessing $O(n)$, query w $O(\log n \cdot T(n))$, gdzie $T(n)$ to czas operacji na przedziale bazowym. Pamięć $O(n \log n)$.

```
const int base = 1<<20;
struct mergeSortTree
{
  vector<vector<int> > tree;
  void build(int w, vector<int>& A)
  {
    if(w >= base)
    {
      if(w - base < A.size()) tree[w].push_back({A[w-base]});
      return;
    }
    build(w*2, A);
    build(w*2+1, A);
    merge(tree[w*2].begin(), tree[w*2].end(),
    tree[w*2+1].begin(), tree[w*2+1].end(),
    back_inserter(tree[w]));
    return;
  }
  mergeSortTree(vector<int>& A)
  {
    tree.resize(base*2);
    build(1, A);
  }
  int query(int w, int p, int k, int x, int y, int val)
  {
    if(p > y || k < x) return 0;//poza zapytaniem
    if(x <= p && k <= y)
    {
      auto it = upper_bound(tree[w].begin(), tree[w].end(), val
          );
      //tutaj robimy cos na przedziale bazowym zapytania
      return distance(tree[w].begin(), it);//returnujemy wynik
          tego przedzialu
    }
    return query(w*2, p, (p+k)/2, x, y, val) +
    query(w*2+1, (p+k)/2+1, k, x, y, val); //tu dowolna
        operacja (max, +, -, *, czy cokolwiek)
  }
  int query(int x, int y, int val) {return query(1, 0, base-1,
      x, y, val);}//ile mniejszych od val na (x, y)
};
```

online-cht.cpp

Opis: Dodawanie prostej i query w $O(\log n)$. Daje min z funkcji liniowych.

```
struct Line {
  bool type;
  double x;
  ll m, c;
};

bool operator<(Line l1, Line l2) {
  if (l1.type || l2.type) return l1.x < l2.x;
  return l1.m > l2.m;
}

struct CHT{
  set<Line> cht;
  bool has_prev(set<Line>::iterator it) { return it != cht.
      begin(); }
  bool has_next(set<Line>::iterator it) {
    return it != cht.end() && next(it) != cht.end();
  }
  double intersect(set<Line>::iterator l1, set<Line>::iterator
      l2) {
    return (double)(l1->c - l2->c) / (l2->m - l1->m);
  }
  void calc_x(set<Line>::iterator it) {
    if (has_prev(it)) {
      Line l = *it;
      l.x = intersect(prev(it), it);
      cht.insert(cht.erase(it), l);
    }
  }
```

```
  bool bad(set<Line>::iterator it) {
    if (has_next(it) && next(it)->c <= it->c) return true;
    return (has_prev(it) && has_next(it) && intersect(prev(it),
          next(it)) <= intersect(prev(it), it));
  }
  void add_line(ll m, ll c) {
    set<Line>::iterator it;
    it = cht.lower_bound({0, 0, m, c});
    if (it != cht.end() && it->m == m) {
      if (it->c <= c) return;
      cht.erase(it);
    }
    it = cht.insert({0, 0, m, c}).first;
    if (bad(it)) cht.erase(it);
    else {
      while (has_prev(it) && bad(prev(it))) cht.erase(prev(it))
          ;
      while (has_next(it) && bad(next(it))) cht.erase(next(it))
          ;
      if (has_next(it)) calc_x(next(it));
      calc_x(it);
    }
  }
  ll query(ll h) {
    Line l = *prev(cht.upper_bound({1, (double)h, 0, 0}));
    return l.m * h + l.c;
  }
};
```

wavelet.cpp

Opis: $O(\log^2 \text{ALPH})$ czasowo, $O(n \log \text{ALPH})$ pamięciowo. Constructor przyjmuje ciąg na którym rozpinamy drzewo, `rank(l, r, k)` zwraca k-tą najmniejszą liczbę na przedziale $[l; r]$. (**Uwaga!**) pamięć około 262 Mb, dla $N, Q \leq 2 \cdot 10^5$.

```
//funkcja cnt jest w O(\log) co daje cale operacje w O(\log^2),
      da sie cnt w czasie stalym -- do dopisania
struct wavelet_tree {
  const static int ALPH = (1<<18)-1;
  struct Node {
    int low, high, mid;
    vector<int> v;
    Node *left, *right;

    Node() {
      low = 0, high = ALPH;
      left = right = nullptr;
      mid = (low+high)/2;
    }

    Node (const vector<int> &x, const vector<int> &t, int a =
        0, int b = ALPH): low(a), high(b), v(x) {
      mid = ((ll)low+high)/2;
      if(ssize(x) && low != high) {
        vector<int> div[2];
        for(auto e : v) div[t[e]<=mid].eb(e);
        if(ssize(div[1])) left = new Node(div[1], t, low, mid);
        else left = nullptr;
        if(ssize(div[0])) right = new Node(div[0], t, mid+1,
            high);
        else right = nullptr;
      }
      else {
        left = right = nullptr;
      }
    }
  };

  Node *root;
  wavelet_tree(const vector<int> &b) {
    vector<int> idx;
    REP(i, ssize(b)) idx.eb(i);
    root = new Node(idx, b);
  };

  int cnt(Node *v, int l, int r) {
    if(v == nullptr) return 0;
    return int(upper_bound(all(v->v), r)-lower_bound(all(v->v),
          l));
  }

  int rank(int l, int r, int k, Node *v) {
    if(v->low == v->high) return v->low;
    if(cnt(v->left, l, r) >= k) return rank(l, r, k, v->left);
    else return rank(l, r, k-cnt(v->left, l, r), v->right);
  }
  int rank(int l, int r, int k) {
    return rank(l, r, k, root);
  }

  void clear(Node *v) {
    if(v->left != nullptr) clear(v->left);
    if(v->right != nullptr) clear(v->right);
    delete v;
  }
  ~wavelet_tree() {
    clear(root);
  }
};
```

Grafy (5)

5.1 Drzewo DFS

Mamy drzewo dfs z obliczonymi czasami wejścia w tablicy **pre** i funkcją

$$\mathbf{low}[v] = \min \begin{cases} \mathbf{pre}[v], \\ \mathbf{pre}[u] & \text{dla krawędzi niedrzewowych } (u, v), \\ \mathbf{low}[u] & \text{dla krawędzi drzewowych } (u, v). \end{cases}$$

Wierzchołek v jest punktem artykulacji jeżeli

$$\mathbf{low}[u] \geq \mathbf{pre}[v]$$

dla dowolnego u będącego synem v w drzewie dfs.

Krawędź (v, u) (v jest ojcem u) jest mostem, jeżeli jest krawędzią drzewową i

$$\mathbf{low}[u] > \mathbf{pre}[v].$$

hld.cpp

Opis: $O(n)$ preprocessing, update i query w $O(\log^2 n)$. Update to zmiana wartosci w wierzcholku, query to max na sciezce. Drzewo przedziałowe do dopisania tak jak w komentarzu.

```
const int inf = 1e9+10;

//struct segtree {
//  segtree(int n = 0) {}
//  void update(int v, int x) {}
//  int query(int l, int r) {}
//};

struct HLD {
  int n;
  vector<vector<int>> g;
  segtree t;
  int timer;

  vector<int> sz, d, parent, path, c, pre;

  HLD(const vector<vector<int>> &graph): n(ssize(graph)), g(
      graph) {
    sz = d = parent = path = c = pre = vector<int> (n);

    d[0] = 0;
    dfs(0);

    path[0] = 0;
    timer = 0;
    make_hld(0);
    t = segtree(timer);
  }

  void dfs(int v, int p = -1) {
    parent[v] = p;
    sz[v] = 1;
    c[v] = -1;
    for (auto u : g[v]) {
      if (u == p) continue;
      d[u] = d[v] + 1;
      dfs(u, v);
      sz[v] += sz[u];
      if (c[v] == -1 || sz[c[v]] < sz[u]) c[v] = u;
    }
  }

  void make_hld(int v, int p = -1) {
    pre[v] = timer++;
    if (c[v] != -1) {
      path[c[v]] = path[v];
      make_hld(c[v], v);
    }
    for (auto u : g[v]) {
      if (u == p || u == c[v]) continue;
      path[u] = u;
      make_hld(u, v);
    }
  }

  void update(int v, int x) {
    t.update(pre[v], x);
  }

  int query(int a, int b) {
    int res = -inf;
    while (path[a] != path[b]) {
      if (d[path[a]] < d[path[b]]) swap(a, b);
      res = max(res, t.query(pre[path[a]], pre[a]));
      a = parent[path[a]];
    }
    if (d[a] < d[b]) swap(a, b);
    res = max(res, t.query(pre[b], pre[a]));
    return res;
  }
};
```

scc.cpp

Opis: silnie spójne składowe. Argument do constructora to graf skierowany w postaci vector<vector<int>> v. Oblicza vector indeksów spójnie składowej do której należy v o nazwie scc. Czas, pamiec $O(n+m)$.

```
struct SCC{
  vector<vector<int>>v,x;
  int n;
  queue<int>q;
  vector<bool>vis;
  int kt=0;
  vector<int>scc;
  SCC(const vector<vector<int>>&vv):v(vv),n(ssize(vv)){
    vis.resize(n,0);
    scc.resize(n);
    for(int i=0;i<n;i++){
      if(!vis[i])dfs0(i);
    }
    make_x();
    fill(all(vis),0);
    while(!q.empty()){
      if(!vis[q.front()]){
        dfs1(q.front());
        kt++;
      }
      q.pop();
    }
  }
  void dfs0(int a){
    vis[a]=1;
    for(auto b:v[a]){
      if(!vis[b])dfs0(b);
    }
    q.push(a);
  }
  void make_x(){
    x.resize(n);
    for(int i=0;i<ssize(v);i++){
      for(auto b:v[i]){
        x[b].push_back(i);
      }
    }
  }
  void dfs1(int a){
    vis[a]=1;
    scc[a]=kt;
    for(auto b:v[a]){
      if(!vis[b])dfs1(b);
    }
  }
};
```

bellman-ford.cpp

Opis: W $O(nm)$ oblicza tablice **D** z odległościami od s w grafie skierowanym ważonym (dopuszczane ujemne wagi). Po obliczeniu odległości isNegCyc() zwraca czy w grafie jest ujemny cykl.

```
const int INF = numeric_limits<int>::max();
struct edge
{
  int a, b, cost;
  edge(int _a, int _b, int _cost){a = _a; b = _b; cost = _cost;
      }
};
struct Bellman_Ford
{
  int n;
  vector<edge> edges;
  vector<int> D;
  bool makeStep()
  {
    bool e = 0;
    for(auto v : edges)
      if(D[v.a] != INF)
        if(D[v.b] > D[v.a] + v.cost)
          D[v.b] = D[v.a] + v.cost, e = true;
    return e;
  }
  Bellman_Ford(int _n, vector<edge>& _edges, int s)
  {
    n = _n; edges = _edges;
    D.resize(n+1, INF);
    D[s] = 0;
    while(_n-->1) makeStep();
  }
  bool isNegCyc()
  {
    return makeStep();
  }
};
```

5.2 Floyd-Warshall

$O(n^3)$. Oblicza odległość między każdą parą wierzchołków w skierowanym grafie ważonym (dopuszczalne wagi ujemne).

```
Dla k = 0 do n - 1
    Dla i = 0 do n - 1
        Dla j = 0 do n - 1
            Odleglosc[i, j] = min(Odleglosc[i, j],
     Odleglosc[i, k] + Odleglosc[k, j])
```

Tekstówki (6)

kmp.cpp

Opis: $O(n+m)$. Constructor przyjmuje wzorzec i tekst, oblicza vector indeksów wystąpień wzorca w tekście.

```
struct KMP{
  string s;
  vector<int>pi;
  vector<int>ans;
  KMP(string &a,string &b):s(a+'#'+b){
    prefsuf();
    for(int i=ssize(a);i<ssize(s);i++){
      if(pi[i]==ssize(a))ans.push_back(i-2*ssize(a));
    }
  }
  void prefsuf(){
    int n=ssize(s);
    pi.resize(n,0);
    for(int i=1;i<n;i++){
      int j=pi[i-1];
      while(j>0&&s[i]!=s[j])j=pi[j-1];
      pi[i]=j+(s[i]==s[j]);
    }
  }
};
```

z.cpp

Opis: $O(n)$. Zwraca tablice z, gdzie z[i] jest równe najdłuższemu wspólnemu prefiksowi słowa s i sufiksu $[i,\ldots,n-1]$.

```
vector<int> z(string s) {
  int n = ssize(s);
  vector<int> z(n);
  int l = 0, r = 0;
  for (int i = 1; i < n; ++i) {
    if (i < r) z[i] = min(z[i-l], r-i);
    while (i + z[i] < n && s[i+z[i]] == s[z[i]]) ++z[i];
    if (i + z[i] > r) {
      r = i + z[i];
      l = i;
    }
  }
  return z;
}
```

suffix-array.cpp

Opis: (źródło: https://github.com/tonowak/acmlib/blob/master/code/string/su
array-short/main.cpp) $O(n \log n)$, zawiera posortowane suffixy, zawiera pusty suffix, $lcp[i]$ to lcp suffixu $sa[i-1]$ i $sa[i]$, Dla s = aabaaab, sa={7,3,4,0,5,1,6,2},lcp={0,0,2,3,1,2,0,1}

```
pair<vector<int>, vector<int>> suffix_array(vector<int> s, int
     alpha = 26) {
  ++alpha;
  for(int &c : s) ++c;
  s.emplace_back(0);
  int n = ssize(s), k = 0, a, b;
  vector<int> x(s.begin(), s.end());
  vector<int> y(n), ws(max(n, alpha)), rank(n);
  vector<int> sa = y, lcp = y;
  iota(sa.begin(), sa.end(), 0);
  for(int j = 0, p = 0; p < n; j = max(1, j * 2), alpha = p) {
    p = j;
    iota(y.begin(), y.end(), n - j);
    for (int i = 0; i < n; ++i) if(sa[i] >= j)
      y[p++] = sa[i] - j;
    fill(ws.begin(), ws.end(), 0);
    for (int i = 0; i < n; ++i) ws[x[i]]++;
    for (int i = 1; i <= alpha - 1; ++i) ws[i] += ws[i - 1];
    for(int i = n; i--;) sa[--ws[x[y[i]]]] = y[i];
    swap(x, y);
    p = 1, x[sa[0]] = 0;
    for (int i = 1; i <= n-1; ++i) a = sa[i - 1], b = sa[i], x[
        b] =
      (y[a] == y[b] && y[a + j] == y[b + j]) ? p - 1 : p++;
  }
  for(int i = 1; i <= n-1; ++i) rank[sa[i]] = i;
  for(int i = 0, j; i < n - 1; lcp[rank[i++]] = k)
    for(k && k--, j = sa[rank[i] - 1];
      s[i + k] == s[j + k]; k++);
  lcp.erase(lcp.begin());
  return {sa, lcp};
}
```

Flow (7)

dinic.cpp

Opis: $O(V^2E)$, chyba że mamy sieć jednostkową (max flow na krawędzi to 1) to $O(E\sqrt{V})$.

```
const int INF = numeric_limits<int>::max();
struct flowEdge
{
  int u, v, cap, flow;
  flowEdge(int _u, int _v, int _cap, int _flow){u = _u; v = _v;
       cap = _cap; flow = _flow;}
  flowEdge(){}
};
struct dinic
{
  vector<flowEdge> edges;
```

```
  vector<vector<int> > idx;
  queue<int> q;
  vector<int> lvl;
  vector<int> ptr;
  int n, m;
  dinic(int _n, int _m, vector<pair<pair<int, int>, int> >&
       _edges) //krawedzie musza byc znormalizowane do DG
  {
    n = _n; m = _m;
    idx.resize(n+1);
    lvl.resize(n+1);
    ptr.resize(n+1);
    for(auto v : _edges)
    {
      edges.push_back(flowEdge(v.first.first, v.first.second, v
           .second, 0));
      edges.push_back(flowEdge(v.first.second, v.first.first,
           0, 0));
      idx[v.first.first].push_back(edges.size()-2);
      idx[v.first.second].push_back(edges.size()-1);
    }
  }
  bool bfs()
  {
    fill(lvl.begin(), lvl.end(), -1);
    q.push(0);
    lvl[0] = 0;
    while(!q.empty())
    {
      int a = q.front();
      q.pop();
      for(auto v : idx[a])
      {
        if(lvl[edges[v].v] != -1) continue;
        if(edges[v].cap - edges[v].flow < 1) continue;
        lvl[edges[v].v] = lvl[a]+1;
        q.push(edges[v].v);
      }
    }
    return lvl[n] != -1;
  }
  int dfs(int v, int aktmin)
  {
    if(v == n) return aktmin;
    if(aktmin == 0) return aktmin;
    for(int& i = ptr[v];i<idx[v].size();i++)
    {
      int id = idx[v][i];
      int nv = edges[id].v;
      if(lvl[v] + 1 != lvl[nv] || edges[id].cap - edges[id].
           flow < 1) continue;
      long long bFlow = dfs(nv, min(aktmin, edges[id].cap -
           edges[id].flow));
      if(bFlow == 0) continue;
      edges[id].flow += bFlow;
      edges[id^1].flow -= bFlow;
      return bFlow;
    }
    return 0;
  }
  int maxFlow()
  {
    int res = 0;
    while(true)
    {
      if(!bfs()) break;
      fill(ptr.begin(), ptr.end(), 0);
      while(int flow = dfs(0, INF)) res += flow;
    }
    return res;
  }
};
```

Bignumy (8)

big-num.cpp

Opis: bignumy.

```
typedef long long ll;
const int maxn = 1e2 + 14, lg = 15;

const int base = 1000000000;
const int base_digits = 9;
struct bigint {
  vector<int> a;
  int sign;
  /*<arpa>*/
  int size(){
    if(a.empty())return 0;
    int ans=(a.size()-1)*base_digits;
    int ca=a.back();
    while(ca)
      ans++,ca/=10;
    return ans;
  }
  bigint operator ^(const bigint &v){
    bigint ans=1,a=*this,b=v;
    while(!b.isZero()){
      if(b%2)
        ans*=a;
      a*=a,b/=2;
    }
    return ans;
  }
  string to_string(){
    stringstream ss;
    ss << *this;
    string s;
    ss >> s;
    return s;
  }
  int sumof(){
    string s = to_string();
    int ans = 0;
    for(auto c : s)  ans += c - '0';
    return ans;
  }
  /*</arpa>*/
  bigint() :
    sign(1) {
  }

  bigint(long long v) {
    *this = v;
  }

  bigint(const string &s) {
    read(s);
  }

  void operator=(const bigint &v) {
    sign = v.sign;
    a = v.a;
  }

  void operator=(long long v) {
    sign = 1;
    a.clear();
    if (v < 0)
      sign = -1, v = -v;
    for (; v > 0; v = v / base)
      a.push_back(v % base);
  }

  bigint operator+(const bigint &v) const {
    if (sign == v.sign) {
      bigint res = v;

      for (int i = 0, carry = 0; i < (int) max(a.size(), v.a.
          size()) || carry; ++i) {
        if (i == (int) res.a.size())
          res.a.push_back(0);
        res.a[i] += carry + (i < (int) a.size() ? a[i] : 0);
        carry = res.a[i] >= base;
        if (carry)
          res.a[i] -= base;
      }
      return res;
    }
    return *this - (-v);
  }

  bigint operator-(const bigint &v) const {
    if (sign == v.sign) {
      if (abs() >= v.abs()) {
        bigint res = *this;
        for (int i = 0, carry = 0; i < (int) v.a.size() ||
            carry; ++i) {
          res.a[i] -= carry + (i < (int) v.a.size() ? v.a[i] :
              0);
          carry = res.a[i] < 0;
          if (carry)
            res.a[i] += base;
        }
        res.trim();
        return res;
      }
      return -(v - *this);
    }
    return *this + (-v);
  }

  void operator*=(int v) {
    if (v < 0)
      sign = -sign, v = -v;
    for (int i = 0, carry = 0; i < (int) a.size() || carry; ++i
        ) {
      if (i == (int) a.size())
        a.push_back(0);
      long long cur = a[i] * (long long) v + carry;
      carry = (int) (cur / base);
      a[i] = (int) (cur % base);
      //asm("divl %%ecx" : "=a"(carry), "=d"(a[i]) : "A"(cur),
          "c"(base));
    }
    trim();
  }

  bigint operator*(int v) const {
    bigint res = *this;
    res *= v;
    return res;
  }

  void operator*=(long long v) {
    if (v < 0)
      sign = -sign, v = -v;
    if(v > base){
      *this = *this * (v / base) * base + *this * (v % base);
      return ;
    }
    for (int i = 0, carry = 0; i < (int) a.size() || carry; ++i
        ) {
      if (i == (int) a.size())
        a.push_back(0);
      long long cur = a[i] * (long long) v + carry;
      carry = (int) (cur / base);
      a[i] = (int) (cur % base);
      //asm("divl %%ecx" : "=a"(carry), "=d"(a[i]) : "A"(cur),
          "c"(base));
    }
    trim();
  }

  bigint operator*(long long v) const {
    bigint res = *this;
    res *= v;
    return res;
  }

  friend pair<bigint, bigint> divmod(const bigint &a1, const
      bigint &b1) {
    int norm = base / (b1.a.back() + 1);
    bigint a = a1.abs() * norm;
    bigint b = b1.abs() * norm;
    bigint q, r;
    q.a.resize(a.a.size());

    for (int i = a.a.size() - 1; i >= 0; i--) {
      r *= base;
      r += a.a[i];
      int s1 = r.a.size() <= b.a.size() ? 0 : r.a[b.a.size()];
      int s2 = r.a.size() <= b.a.size() - 1 ? 0 : r.a[b.a.size
          () - 1];
      int d = ((long long) base * s1 + s2) / b.a.back();
      r -= b * d;
      while (r < 0)
        r += b, --d;
      q.a[i] = d;
    }

    q.sign = a1.sign * b1.sign;
    r.sign = a1.sign;
    q.trim();
    r.trim();
    return make_pair(q, r / norm);
  }

  bigint operator/(const bigint &v) const {
    return divmod(*this, v).first;
  }

  bigint operator%(const bigint &v) const {
    return divmod(*this, v).second;
  }

  void operator/=(int v) {
    if (v < 0)
      sign = -sign, v = -v;
    for (int i = (int) a.size() - 1, rem = 0; i >= 0; --i) {
      long long cur = a[i] + rem * (long long) base;
      a[i] = (int) (cur / v);
      rem = (int) (cur % v);
    }
    trim();
  }

  bigint operator/(int v) const {
    bigint res = *this;
    res /= v;
```

```
  return res;
}

int operator%(int v) const {
  if (v < 0)
    v = -v;
  int m = 0;
  for (int i = a.size() - 1; i >= 0; --i)
    m = (a[i] + m * (long long) base) % v;
  return m * sign;
}

void operator+=(const bigint &v) {
  *this = *this + v;
}
void operator-=(const bigint &v) {
  *this = *this - v;
}
void operator*=(const bigint &v) {
  *this = *this * v;
}
void operator/=(const bigint &v) {
  *this = *this / v;
}

bool operator<(const bigint &v) const {
  if (sign != v.sign)
    return sign < v.sign;
  if (a.size() != v.a.size())
    return a.size() * sign < v.a.size() * v.sign;
  for (int i = a.size() - 1; i >= 0; i--)
    if (a[i] != v.a[i])
      return a[i] * sign < v.a[i] * sign;
  return false;
}

bool operator>(const bigint &v) const {
  return v < *this;
}
bool operator<=(const bigint &v) const {
  return !(v < *this);
}
bool operator>=(const bigint &v) const {
  return !(*this < v);
}
bool operator==(const bigint &v) const {
  return !(*this < v) && !(v < *this);
}
bool operator!=(const bigint &v) const {
  return *this < v || v < *this;
}

void trim() {
  while (!a.empty() && !a.back())
    a.pop_back();
  if (a.empty())
    sign = 1;
}

bool isZero() const {
  return a.empty() || (a.size() == 1 && !a[0]);
}

bigint operator-() const {
  bigint res = *this;
  res.sign = -sign;
  return res;
}

bigint abs() const {
  bigint res = *this;
  res.sign *= res.sign;
  return res;
}

long long longValue() const {
  long long res = 0;
  for (int i = a.size() - 1; i >= 0; i--)
    res = res * base + a[i];
  return res * sign;
}

friend bigint gcd(const bigint &a, const bigint &b) {
  return b.isZero() ? a : gcd(b, a % b);
}
friend bigint lcm(const bigint &a, const bigint &b) {
  return a / gcd(a, b) * b;
}

void read(const string &s) {
  sign = 1;
  a.clear();
  int pos = 0;
  while (pos < (int) s.size() && (s[pos] == '-' || s[pos] ==
      '+')) {
    if (s[pos] == '-')
      sign = -sign;
    ++pos;
  }
  for (int i = s.size() - 1; i >= pos; i -= base_digits) {
    int x = 0;
    for (int j = max(pos, i - base_digits + 1); j <= i; j++)
      x = x * 10 + s[j] - '0';
    a.push_back(x);
  }
  trim();
}

friend istream& operator>>(istream &stream, bigint &v) {
  string s;
  stream >> s;
  v.read(s);
  return stream;
}

friend ostream& operator<<(ostream &stream, const bigint &v)
    {
  if (v.sign == -1)
    stream << '-';
  stream << (v.a.empty() ? 0 : v.a.back());
  for (int i = (int) v.a.size() - 2; i >= 0; --i)
    stream << setw(base_digits) << setfill('0') << v.a[i];
  return stream;
}

static vector<int> convert_base(const vector<int> &a, int
    old_digits, int new_digits) {
  vector<long long> p(max(old_digits, new_digits) + 1);
  p[0] = 1;
  for (int i = 1; i < (int) p.size(); i++)
    p[i] = p[i - 1] * 10;
  vector<int> res;
  long long cur = 0;
  int cur_digits = 0;
  for (int i = 0; i < (int) a.size(); i++) {
    cur += a[i] * p[cur_digits];
    cur_digits += old_digits;
    while (cur_digits >= new_digits) {
      res.push_back(int(cur % p[new_digits]));
      cur /= p[new_digits];
      cur_digits -= new_digits;
    }
  }
  res.push_back((int) cur);
  while (!res.empty() && !res.back())
    res.pop_back();
  return res;
}

typedef vector<long long> vll;
static vll karatsubaMultiply(const vll &a, const vll &b) {
  int n = a.size();
  vll res(n + n);
  if (n <= 32) {
    for (int i = 0; i < n; i++)
      for (int j = 0; j < n; j++)
        res[i + j] += a[i] * b[j];
    return res;
  }
  int k = n >> 1;
  vll a1(a.begin(), a.begin() + k);
  vll a2(a.begin() + k, a.end());
  vll b1(b.begin(), b.begin() + k);
  vll b2(b.begin() + k, b.end());
  vll a1b1 = karatsubaMultiply(a1, b1);
  vll a2b2 = karatsubaMultiply(a2, b2);
  for (int i = 0; i < k; i++)
    a2[i] += a1[i];
  for (int i = 0; i < k; i++)
    b2[i] += b1[i];
  vll r = karatsubaMultiply(a2, b2);
  for (int i = 0; i < (int) a1b1.size(); i++)
    r[i] -= a1b1[i];
  for (int i = 0; i < (int) a2b2.size(); i++)
    r[i] -= a2b2[i];
  for (int i = 0; i < (int) r.size(); i++)
    res[i + k] += r[i];
  for (int i = 0; i < (int) a1b1.size(); i++)
    res[i] += a1b1[i];
  for (int i = 0; i < (int) a2b2.size(); i++)
    res[i + n] += a2b2[i];
  return res;
}
bigint operator*(const bigint &v) const {
  vector<int> a6 = convert_base(this->a, base_digits, 6);
  vector<int> b6 = convert_base(v.a, base_digits, 6);
  vll a(a6.begin(), a6.end());
  vll b(b6.begin(), b6.end());
  while (a.size() < b.size())
    a.push_back(0);
  while (b.size() < a.size())
    b.push_back(0);
  while (a.size() & (a.size() - 1))
    a.push_back(0), b.push_back(0);
  vll c = karatsubaMultiply(a, b);
  bigint res;
  res.sign = sign * v.sign;
  for (int i = 0, carry = 0; i < (int) c.size(); i++) {
    long long cur = c[i] + carry;
    res.a.push_back((int) (cur % 1000000));
    carry = (int) (cur / 1000000);
  }
  res.a = convert_base(res.a, 6, base_digits);
  res.trim();
  return res;
}
};
```